UNIT-I

Introduction to AI

In today's world, technology is growing very fast, and we are getting in touch with different new technologies day by day.

Here, one of the booming technologies of computer science is Artificial Intelligence which is ready to create a new revolution in the world by making intelligent machines. Artificial Intelligence is now all around us. It is currently working with a variety of subfields, ranging from general to specific, such as self-driving cars, playing chess, proving theorems, playing music, Painting, etc.

AI is one of the fascinating and universal fields of Computer Science which has a great scope in future. AI holds a tendency to cause a machine to work as a human.

Artificial Intelligence is composed of two words **Artificial** and **Intelligence**, where Artificial defines *"man-made"* i.e., made or produced by human being rather than occurring naturally or a copy of something like naturally. and intelligence defines *"thinking power"* i.e., it is defined as the ability to learn, understand, and apply knowledge and skills. hence AI means *"a man-made thinking power."*

So, we can define AI as:

"It is a branch of computer science by which we can create intelligent machines which can behave like a human, think like humans, and able to make decisions."

Or

Artificial intelligence (AI) is the simulation of human intelligence in machines that are programmed to think and act like humans. Learning,

reasoning, problem-solving, perception, and language comprehension are all examples of cognitive abilities.

Artificial Intelligence is a method of making a computer, a computer-controlled robot, or a software think intelligently like the human mind. AI is accomplished by studying the patterns of the human brain and by analysing the cognitive process. The outcome of these studies develops intelligent software and systems.

Artificial Intelligence exists when a machine can have human based skills such as learning, reasoning, and solving problems.

With Artificial Intelligence you do not need to pre-program a machine to do some work, despite that you can create a machine with programmed algorithms which can work with own intelligence, and that is the awesomeness of AI.

For example, Natural Language Processing (NLP) uses AI to analyze and interpret human language in text or speech. It enables applications like chatbots and virtual assistants to understand user queries, extract meaningful information, and deliver accurate, context-aware responses, transforming unstructured communication into actionable insights.

Core Concepts in AI

Artificial Intelligence (AI) operates on a core set of concepts and technologies that enable machines to perform tasks that typically require human intelligence. Here are some foundational concepts:

1. **Machine Learning (ML)**: This is the backbone of AI, where algorithms learn from data without being explicitly programmed. It involves training an algorithm on a data set, allowing it to improve over time and make predictions or decisions based on new data.

2. **Neural Networks**: Inspired by the human brain, these are networks of algorithms that mimic the way neurons interact, allowing computers to recognize patterns and solve common problems in the fields of AI, machine learning, and deep learning.

3. **Deep Learning**: A subset of ML, deep learning uses complex neural networks with many layers (hence "deep") to analyze various factors of data. This is instrumental in tasks like image and speech recognition.

4. **Natural Language Processing (NLP)**: NLP involves programming computers to process and analyze large amounts of natural language data, enabling interactions between computers and humans using natural language.

5. **Robotics**: While often associated with AI, robotics merges AI concepts with physical components to create machines capable of performing a variety of tasks, from assembly lines to complex surgeries.

6. **Cognitive Computing**: This AI approach mimics human brain processes to solve complex problems, often using pattern recognition, NLP, and data mining.

7. **Expert Systems**: These are AI systems that emulate the decision-making ability of a human expert, applying reasoning capabilities to reach conclusions.

Each of these concepts helps to build systems that can automate, enhance, and sometimes outperform human capabilities in specific tasks.

How Does AI Work?

Artificial intelligence (AI) enables machines to learn from data and recognize patterns in it, to perform tasks more efficiently and effectively. AI works in five steps:

- **Input:** Data is collected from various sources. This data is then sorted into categories.
- **Processing:** The AI sorts and decodes the data using patterns it has been programmed to learn until it recognizes similar patterns in the data.
- **Outcomes:** The AI can then use those patterns to predict outcomes.
- **Adjustments:** If the data sets are considered a "fail," AI learns from that mistake, and the process is repeated under different conditions.
- **Assessments:** In this way, AI is constantly learning and improving.

Why Artificial Intelligence?

Before Learning about Artificial Intelligence, we should know that what is the importance of AI and why should we learn it. Following are some main reasons to learn about AI:

- With the help of AI, you can create such software or devices which can solve real-world problems very easily and with accuracy such as health issues, marketing, traffic issues, etc.
- With the help of AI, you can create your personal virtual Assistant, such as Cortana, Google Assistant, Siri, etc.
- With the help of AI, you can build such Robots which can work in an environment where survival of humans can be at risk.
- AI opens a path for other new technologies, new devices, and new Opportunities.

Goals of Artificial Intelligence:

Following are the main goals of Artificial Intelligence:

1. Replicate human intelligence
2. Solve Knowledge-intensive tasks
3. An intelligent connection of perception and action
4. Building a machine which can perform tasks that requires human intelligence such as:
    - Proving a theorem
    - Playing chess
    - Plan some surgical operation
    - Driving a car in traffic
5. Creating some system which can exhibit intelligent behavior, learn new things by itself, demonstrate, explain, and can advise to its user.

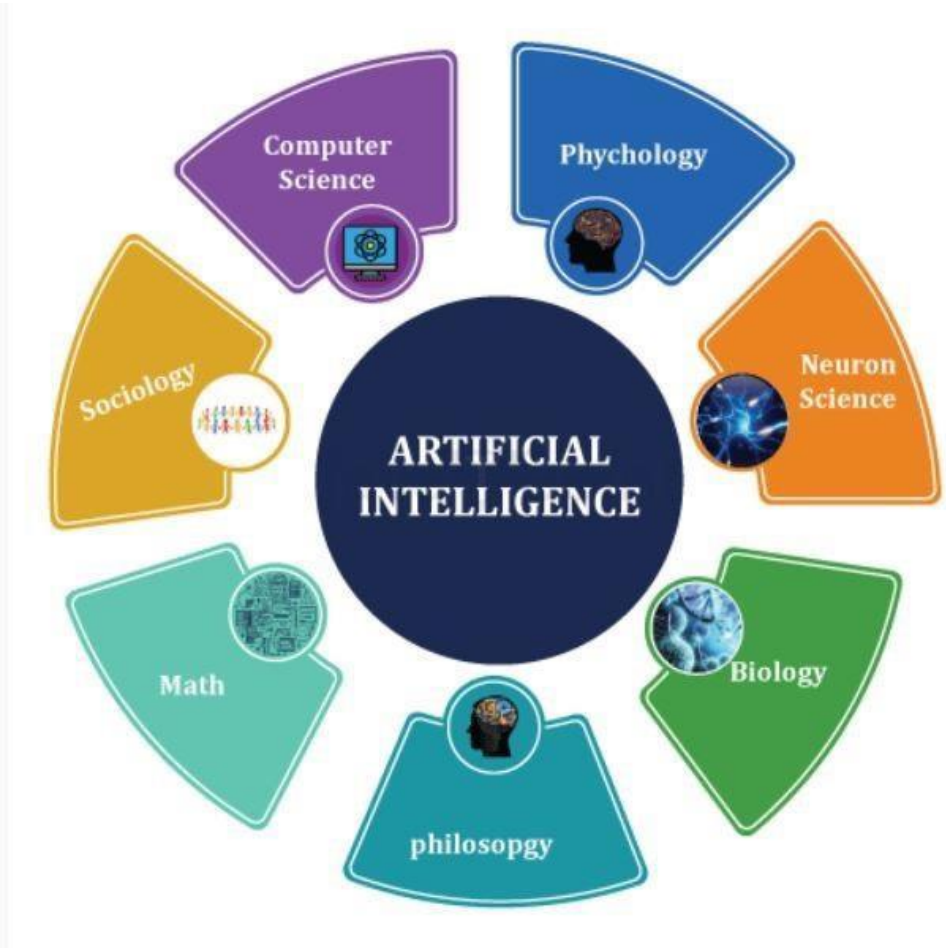
What Comprises to Artificial Intelligence?

Artificial Intelligence is not just a part of computer science even it's so vast and requires lots of other factors which can contribute to it. To create the AI first we should know that how intelligence is composed, so the Intelligence is an intangible part of our brain which is a combination of **Reasoning, learning, problem-solving perception, language understanding, etc**.

To achieve the above factors for a machine or software Artificial Intelligence requires the following discipline:

- Mathematics
- Biology

- Psychology
- Sociology
- Computer Science
- Neurons Study
- Statistics



Types of AI (Artificial Intelligence):

When discussing artificial intelligence (AI), it is common to distinguish between three broad categories: weak AI, strong AI and super AI. Let's explore the characteristics of each type:

- **Weak AI (Narrow AI):**

Weak AI refers to AI systems that are designed to perform specific tasks and are limited to those tasks only. These AI systems excel at their designated functions but lack general intelligence.

Examples of weak AI include voice assistants like Siri or Alexa, recommendation algorithms, and image recognition systems. Weak AI operates within predefined boundaries and cannot generalize beyond their specialized domain.

- **Strong AI (General AI):**

Strong AI, also known as general AI, refers to AI systems that possess human-level intelligence or even surpass human intelligence across a wide range of tasks. Strong AI would be capable of understanding, reasoning, learning, and applying knowledge to solve complex problems in a manner like human cognition. However, the development of strong AI is still largely theoretical and has not been achieved to date.

- **Super AI (S AI)**:

Super intelligent AI, or Artificial Superintelligence, represents an AI that not only mimics but significantly surpasses human intelligence across all fields — science, general wisdom, social skills, and more. ASI would be capable of extraordinary problem-solving and creative abilities, far beyond what current human minds can achieve.

Advantages of Artificial Intelligence:

The widespread adoption of Artificial Intelligence (AI) has brought about numerous benefits and advantages across various industries and aspects of our lives. Here are some of the key benefits of AI:

1. **Improved Efficiency and Productivity:** AI-powered systems can perform tasks with greater speed, accuracy, and consistency than humans, leading to improved efficiency and productivity in various industries. This can result in cost savings, reduced errors, and increased output.

2. **Enhanced Decision-Making:** AI algorithms can analyze large amounts of data, identify patterns, and make informed decisions faster than humans. This can be particularly useful in fields such as finance, healthcare, and logistics, where timely and accurate decision-making is critical.

3. **Personalization and Customization:** AI-powered systems can learn from user behavior and preferences to provide personalized recommendations, content, and experiences. This can lead to increased customer satisfaction and loyalty, as well as improved targeting and marketing strategies.

4. **Automation of Repetitive Tasks:** AI can be used to automate repetitive, time-consuming tasks, freeing up human resources to focus on more strategic and creative work. This can lead to cost savings, reduced errors, and improved work-life balance for employees.

5. **Improved Safety and Risk Mitigation:** AI-powered systems can be used to enhance safety in various applications, such as autonomous vehicles, industrial automation, and medical diagnostics. AI algorithms

can also be used to detect and mitigate risks, such as fraud, cybersecurity threats, and environmental hazards.

6. **Advancements in Scientific Research:** AI can assist in scientific research by analysing large datasets, generating hypotheses, and accelerating the discovery of new insights and breakthroughs. This can lead to advancements in fields such as medicine, climate science, and materials science.

7. **Enhanced Human Capabilities:** AI can be used to augment and enhance human capabilities, such as improving memory, cognitive abilities, and decision-making. This can lead to improved productivity, creativity, and problem-solving skills.

Disadvantages of Artificial Intelligence:

Every technology has some disadvantages, and the same goes for Artificial intelligence. Being so advantageous technology still, it has some disadvantages which we need to keep in our mind while creating an AI system. Following are the disadvantages of AI:

- **High Cost:** The hardware and software requirement of AI is very costly as it requires lots of maintenance to meet current world requirements.
- **Can't think out of the box:** Even we are making smarter machines with AI, but still they cannot work out of the box, as the robot will only do that work for which they are trained or programmed.
- **No feelings and emotions:** AI machines can be an outstanding performer, but still it does not have the feeling so it cannot make any kind of emotional attachment with human and may sometime be harmful for users if the proper care is not taken.

- **Increase dependency on machines:** With the increment of technology, people are getting more dependent on devices and hence they are losing their mental capabilities.
- **No Original Creativity:** As humans are so creative and can imagine some new ideas but still AI machines cannot beat this power of human intelligence and cannot be creative and imaginative.
- **Complexity:** Making and keeping AI systems can be very complicated and need a lot of knowledge. This can make it hard for some groups or people to use them.
- **Job Concerns:** As AI gets better, it might take away not just basic jobs but also some skilled ones. This worries people about losing jobs in different fields.

Challenges of AI:

Artificial Intelligence offers incredible advantages, but it also presents some challenges that need to be addressed:

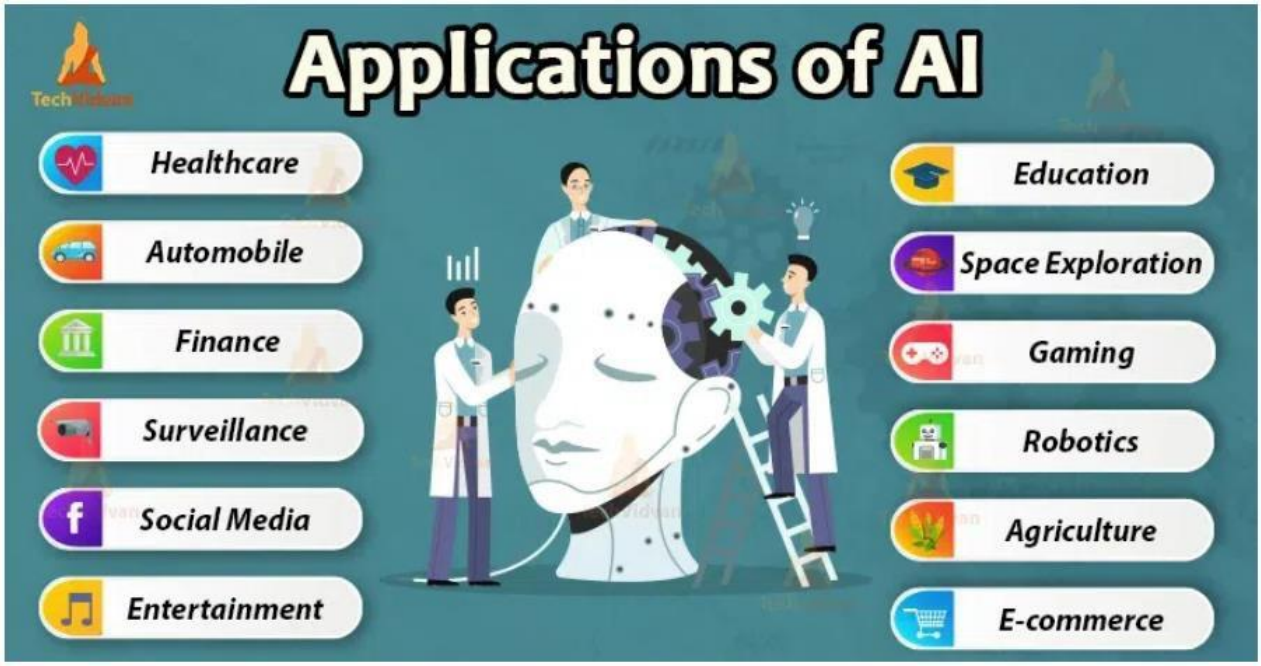
- **Doing the Right Thing:** AI should make the right choices, but sometimes it doesn't. It can make mistakes or do things that aren't fair. We need to teach AI to be better at making good choices.
- **Government and AI:** Sometimes, governments use AI to keep an eye on people. This can be a problem for our freedom. We need to make sure they use AI in a good way.
- **Bias in AI:** AI can sometimes be a bit unfair, especially when it comes to recognizing people's faces. This can cause problems, especially for people who aren't like the majority.

- **AI and social media:** What you see on social media is often decided by AI. But sometimes, AI shows things that aren't true or are kind of mean. We need to make sure AI shows the right stuff.

- **Legal and Regulatory Challenges:** The rapid evolution of AI has outpaced the development of comprehensive laws and regulations, leading to uncertainty about issues like liability and responsibility.

Application of Artificial Intelligence :

Artificial Intelligence has many practical applications across various industries and domains, including:



1. **Healthcare –** AI is used for medical diagnosis by analyzing medical images like X-rays and MRIs to identify diseases. For instance, AI systems are being developed to detect skin cancer from images with high accuracy.

2. **Finance** – AI helps in credit scoring by analysing a borrower's financial history and other data to predict their creditworthiness. This helps banks decide whether to approve a loan and at what interest rate.

3. **Retail** – AI is used for product recommendations by analysing your past purchases and browsing behavior to suggest products you might be interested in. For example, Amazon uses AI to recommend products to customers on their website.

4. **Manufacturing –** AI helps in quality control by inspecting products for defects. AI systems can be trained to identify even very small defects that human inspectors might miss.

5. **Transportation –** AI is used for autonomous vehicles by developing self-driving cars that can navigate roads without human input. Companies like Waymo and Tesla are developing self-driving car technology.

6. **Customer service –** AI-powered chatbots are used to answer customer questions and provide support. For instance, many banks use chatbots to answer customer questions about their accounts and transactions.

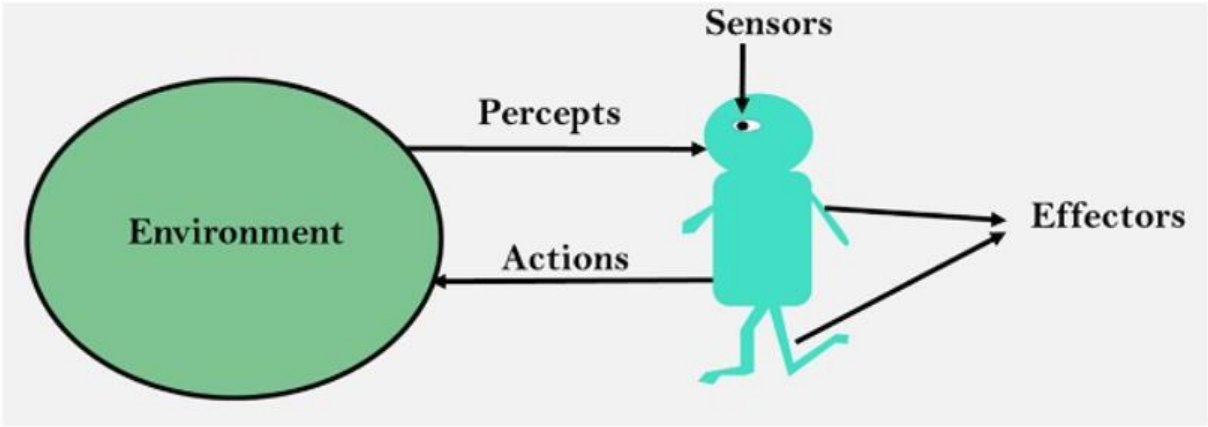
7. **Security –** AI is used for facial recognition by identifying people from images or videos. This technology is used for security purposes, such as identifying criminals or unauthorized individuals.

8. **Marketing –** AI is used for targeted advertising by showing ads to people who are most likely to be interested in the product or service being advertised. For example, social media companies use AI to target ads to users based on their interests and demographics.

9. **Education –** AI is used for personalized learning by tailoring educational content to the individual needs of each student. For example, AI-powered tutoring systems can provide students with personalized instruction and feedback.

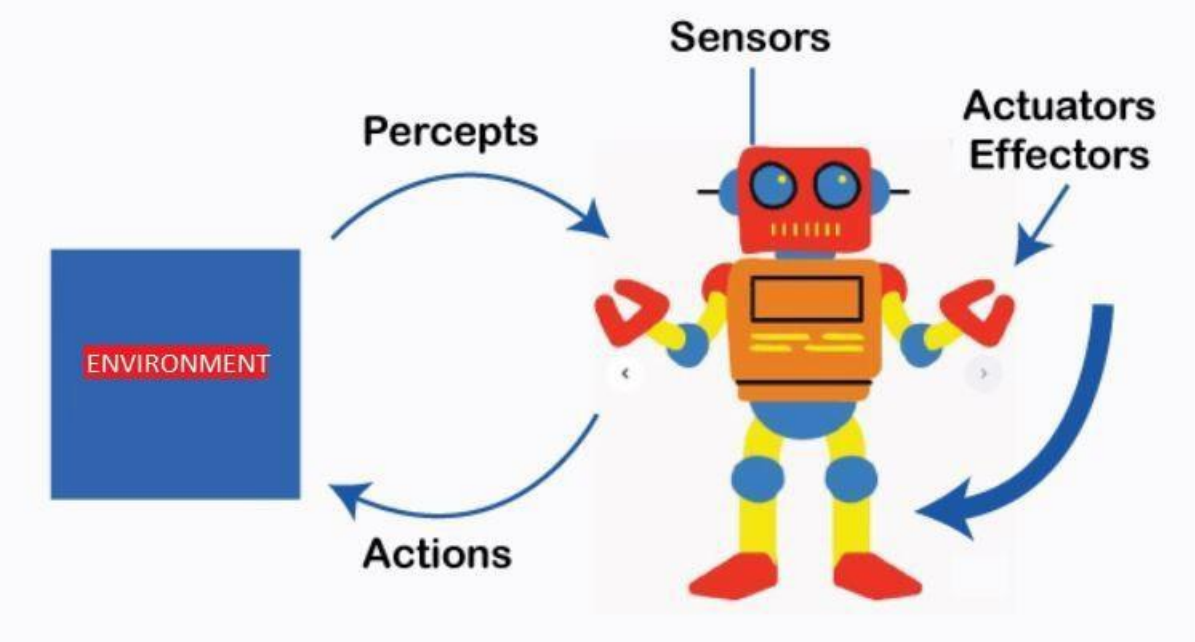
Agents in Artificial Intelligence

An AI system can be defined as the study of the rational agent and its environment. The agents sense the environment through sensors and act on their environment through actuators. An AI agent can have mental properties such as knowledge, belief, intention, etc.



What is an Agent?

An agent can be anything that perceive its environment through sensors and act upon that environment through actuators. An Agent runs in the cycle of perceiving, thinking, and acting.



An agent can be:

- **Human-Agent:** A human agent has eyes, ears, and other organs which work for sensors and hand, legs, vocal tract work for actuators.
- **Robotic Agent:** A robotic agent can have cameras, infrared range finder, NLP for sensors and various motors for actuators.
- **Software Agent:** Software agent can have keystrokes, file contents as sensory input and act on those inputs and display output on the screen.

Hence the world around us is full of agents such as thermostat, cell phone, camera, and even we are also agents.

Before moving forward, we should first know about sensors, effectors, and actuators.

Sensor: Sensor is a device which detects the change in the environment and sends the information to other electronic devices. An agent observes its environment through sensors.

Actuators: Actuators are the component of machines that converts energy into motion. The actuators are only responsible for moving and controlling a system. An actuator can be an electric motor, gears, rails, etc.

Effectors: Effectors are the devices which affect the environment. Effectors can be legs, wheels, arms, fingers, wings, fins, and display screen.

Intelligent Agents

An intelligent agent is an autonomous entity which act upon an environment using sensors and actuators for achieving goals. An intelligent agent may learn from the environment to achieve their goals. A thermostat is an example of an intelligent agent.

Following are the main four rules for an AI agent:

- **Rule 1:** An AI agent must have the ability to perceive the environment.
- **Rule 2:** The observation must be used to make decisions.
- **Rule 3:** Decision should result in an action.
- **Rule 4:** The action taken by an AI agent must be a rational action.

Rational Agent

A rational agent is an agent which has clear preference, models uncertainty, and acts in a way to maximize its performance measure with all possible actions.

A rational agent is said to perform the right things. AI is about creating rational agents to use for game theory and decision theory for various real-world scenarios.

For an AI agent, the rational action is most important because in AI reinforcement learning algorithm, for each best possible action, agent gets the positive reward and for each wrong action, an agent gets a negative reward.

Note: Rational agents in AI are very similar to intelligent agents.

Rationality:

The rationality of an agent is measured by its performance measure. Rationality can be judged based on following points:

- Performance measure which defines the success criterion.
- Agent prior knowledge of its environment.
- Best possible actions that an agent can perform.
- The sequence of percept's.

Note: Rationality differs from Omniscience because an Omniscient agent knows the actual outcome of its action and act accordingly, which is not possible.

Structure of an AI Agent:

The task of AI is to design an agent program which implements the agent function. The structure of an intelligent agent is a combination of architecture and agent program. It can be viewed as:

Agent = Architecture + Agent program

Following are the main three terms involved in the structure of an AI agent:

Architecture: Architecture is machinery that an AI agent executes on.

Agent Function: Agent function is used to map a percept to an action.

$$f:P^* \rightarrow A$$

Agent program: Agent program is an implementation of agent function. An agent program executes on the physical architecture to produce function f.

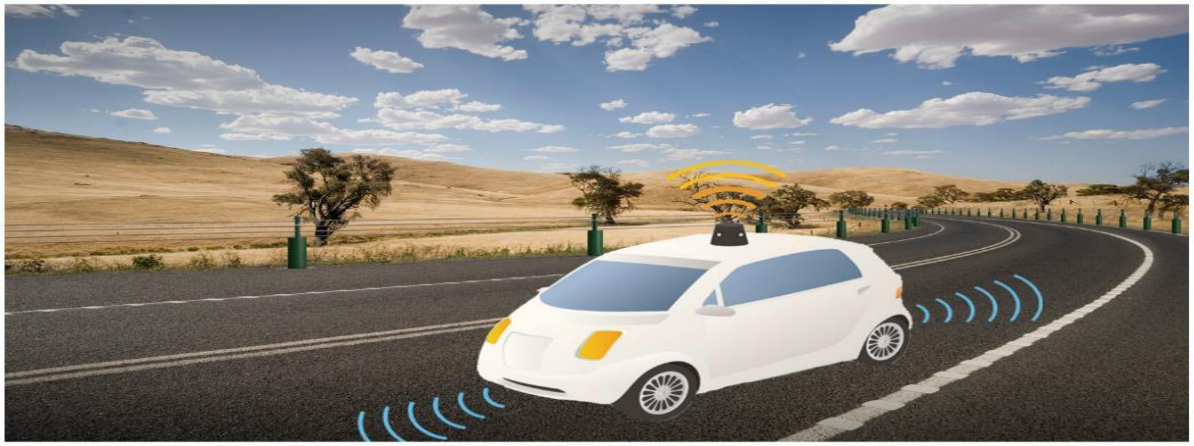
PEAS Representation:

PEAS is a type of model on which an AI agent works upon. When we define an AI agent or rational agent, then we can group its properties under PEAS representation model. It is made up of four words:

- **P:** Performance measure
- **E:** Environment
- **A:** Actuators
- **S:** Sensors

Here performance measure is the objective for the success of an agent's behavior.

PEAS for self-driving cars:



Let's suppose a self-driving car then PEAS representation will be:

Performance: Safety, time, legal drive, comfort

Environment: Roads, other vehicles, road signs, pedestrian

Actuators: Steering, accelerator, brake, signal, horn

Sensors: Camera, GPS, speedometer, odometer, accelerometer, sonar.

Example of Agents with their PEAS representation

Agent	Performance measure	Environment	Actuators	Sensors
1. Medical Diagnose	◦ Healthy patient ◦ Minimized cost	◦ Patient ◦ Hospital ◦ Staff	◦ Tests ◦ Treatments	Keyboard (Entry of symptoms)
2. Vacuum Cleaner	◦ Cleanness ◦ Efficiency ◦ Battery life ◦ Security	◦ Room ◦ Table ◦ Wood floor ◦ Carpet ◦ Various obstacles	◦ Wheels ◦ Brushes ◦ Vacuum Extractor	◦ Camera ◦ Dirt detection sensor ◦ Cliff sensor ◦ Bump Sensor ◦ Infrared Wall Sensor
3. Part -picking Robot	◦ Percentage of parts in correct bins.	◦ Conveyor belt with parts, ◦ Bins	◦ Jointed Arms ◦ Hand	◦ Camera ◦ Joint angle sensors.

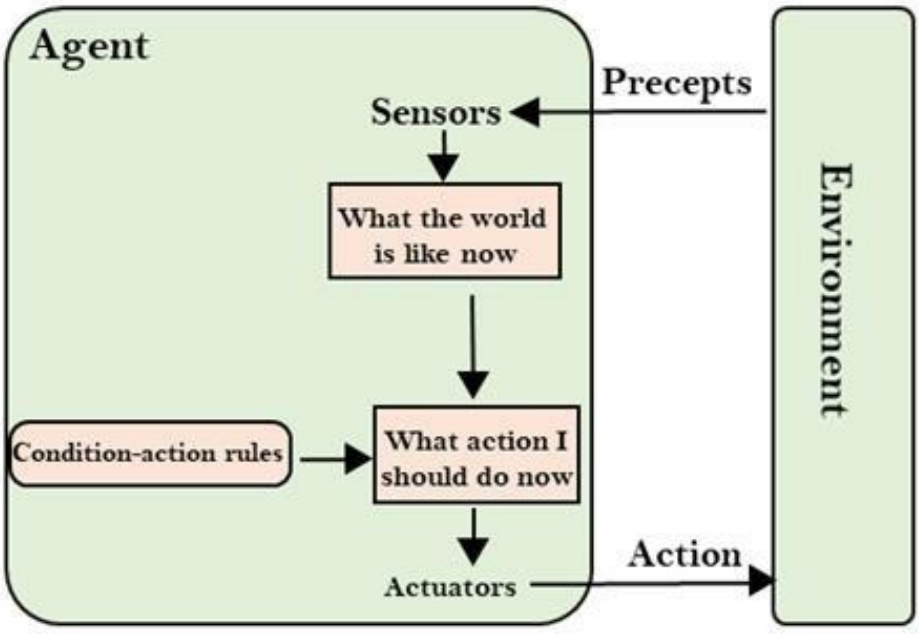
Types of Agents in AI

Agents are often grouped into five classes based on their degree of perceived intelligence and capability. All these agents can improve their performance and generate better action over time. These are given below:

- Simple Reflex Agent
- Model-Based Reflex Agent
- Goal-Based Agents
- Utility-Based Agent
- Learning Agent

Simple Reflex Agent:

- The Simple reflex agents are the simplest agents. These agents take decisions based on the current percepts and ignore the rest of the percept history.
- These agents only succeed in a fully observable environment.
- The Simple reflex agent does not consider any part of percepts history during their decision and action process.
- The Simple reflex agent works on Condition-action rule, which means it maps the current state to action. Such as a Room Cleaner agent, it works only if there is dirt in the room.

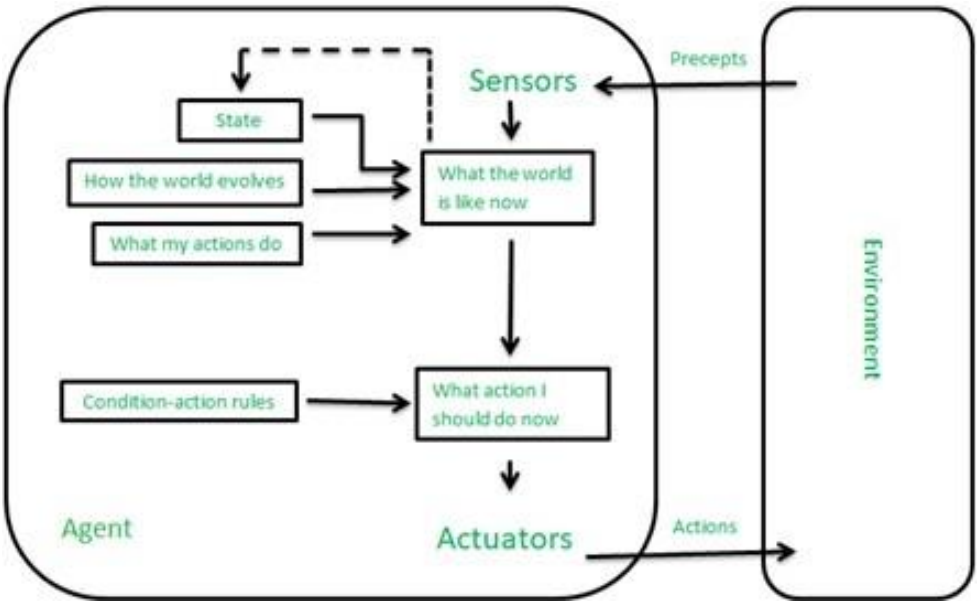


Problems with the straightforward reflex agent design approach:

- They have very limited intelligence.
- They do not have knowledge of non-perceptual parts of the current state.
- Mostly too big to get and to store.
- Not adaptive to changes within the environment.

Model-Based Reflex Agent:

- The Model-based agent can work in a partially observable environment and track the situation.
- A model-based agent has two important factors:
  - **Model:** It is knowledge about "how things happen in the world," so it is called a Model-based agent.
  - **Internal State:** It is a representation of the current state based on percept history.
- These agents have the model, "which is knowledge of the world" and based on the model they perform actions.
- Updating the agent state requires information about:
  a. How the world evolves
  b. How the agent's action affects the world.

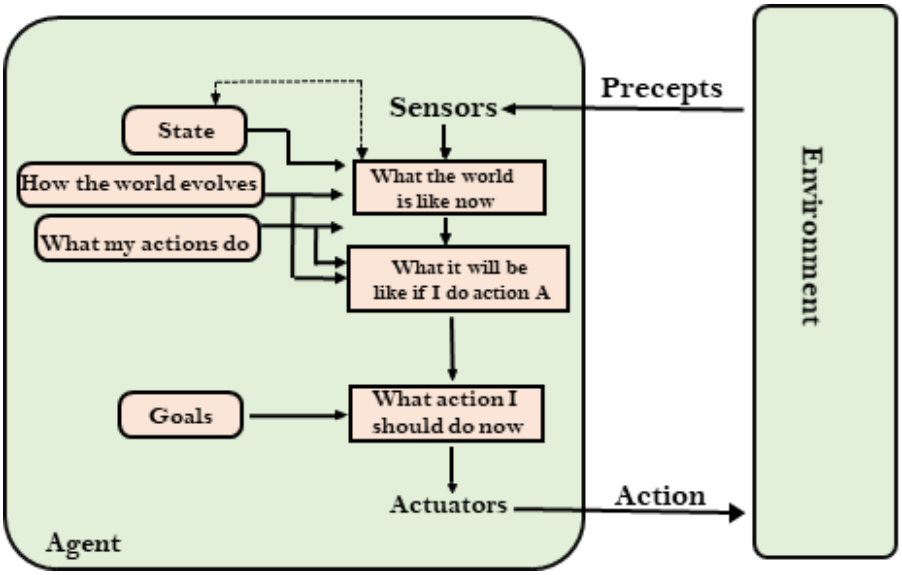


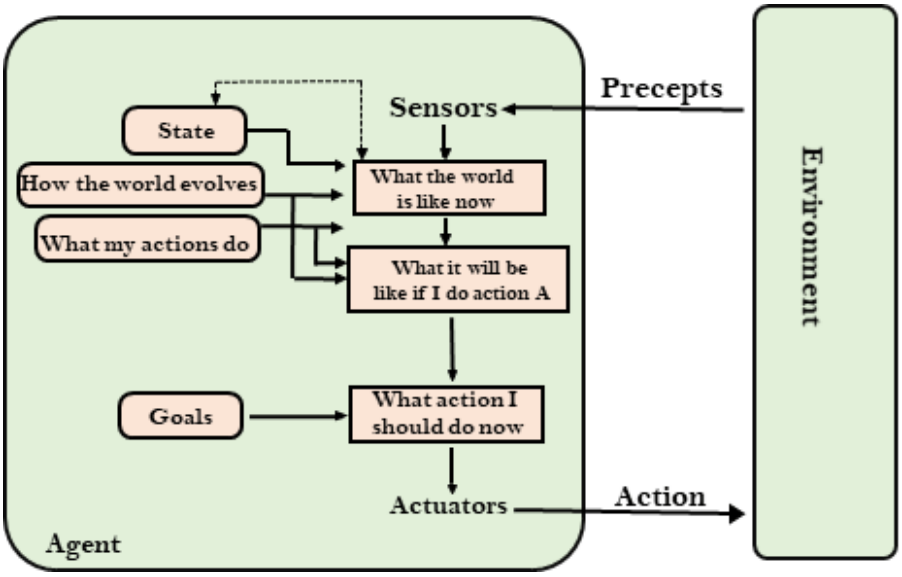
There are some important points that we need to remember about model-based reflex agents:

- Unlike simple reflex agents that rely solely on the current percept, model-based reflex agents consider a broader context.
- It can adapt its behavior based on changes in the environment or new information.
- The ability to reason, plan, and consider a wider context generally leads to improved performance compared to simple reflex agents.

Goal-Based Agent:

- The knowledge of the current state environment is not always sufficient to decide for an agent to what to do.
- The agent needs to know its goal which describes desirable situations.
- Goal-based agents expand the capabilities of the model-based agent by having the "goal" information.
- They choose an action, so that they can achieve the goal.
- These agents may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not. Such considerations of different scenarios are called searching and planning, which makes an agent proactive.



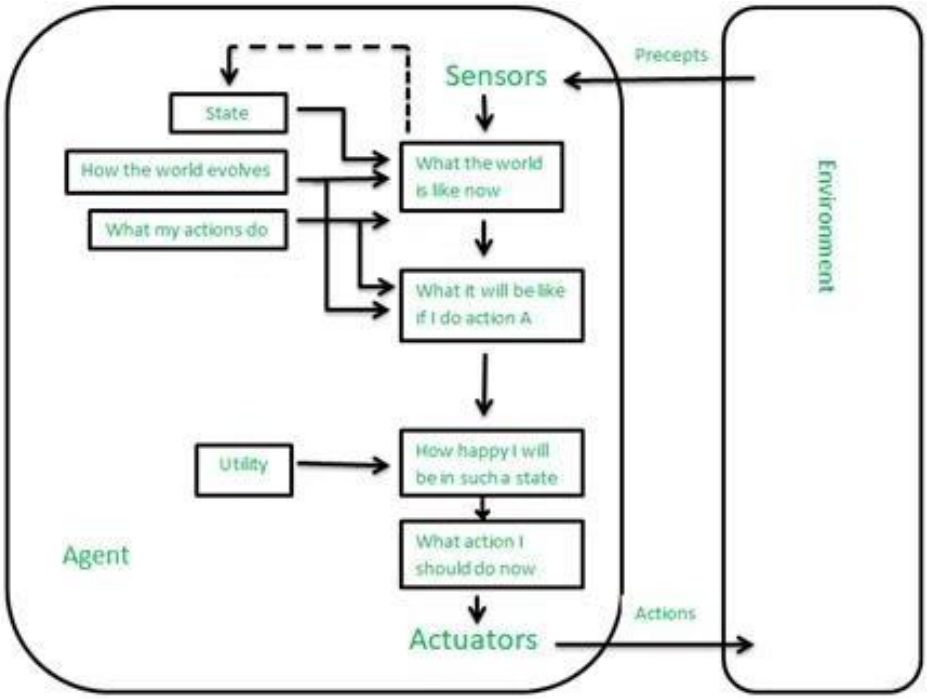

Precepts
Sensors
State
How the world evolves
What my actions do
What the world is like now
What it will be like if I do action A
Goals
What action I should do now
Actuators
Action
Agent
Environment

There are some points about goal-based agent:

- They choose action, in order that they will achieve the goal.
- These agents may need to consider an extended sequence of possible actions before deciding whether the goal is achieved or not.
- Considerations of various scenarios are called searching and planning, which makes an agent proactive.
- Once a plan of action is selected, the goal-based agent executes the actions in the environment.

Utility-Based Agents:

- These agents are like the goal-based agent but provide an extra component of utility measurement which makes them different by providing a measure of success at a given state.
- Utility-based agents act based not only goals but also the best way to achieve the goal.
- The Utility-based agent is useful when there are multiple possible alternatives, and an agent must choose to perform the best action.
- The utility function maps each state to a real number to check how efficiently each action achieves the goals.



There are some points about utility-based agents:

- These agents act based not only on goals but also on the simplest thanks to achieving the goal.

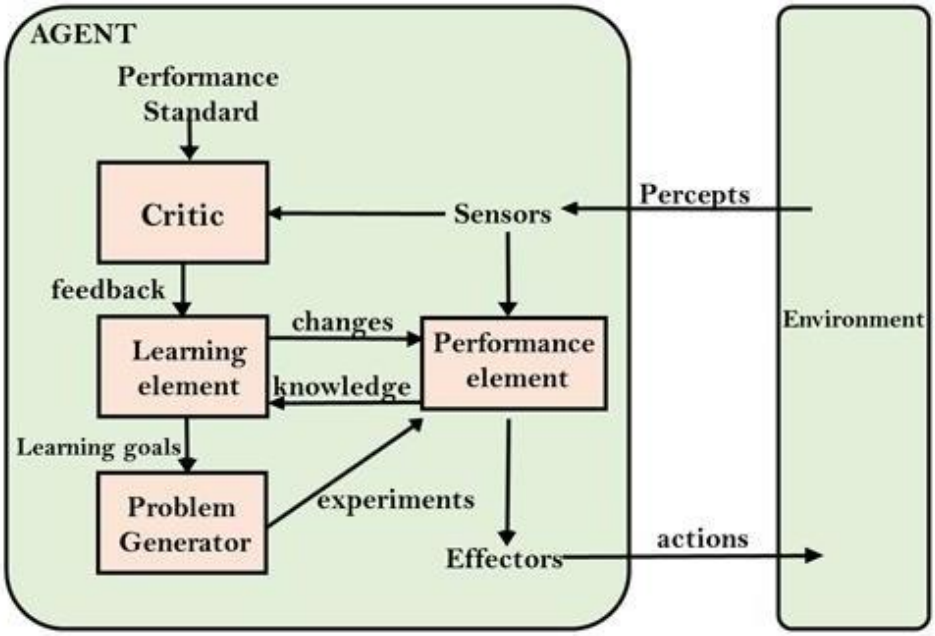
- These agents are beneficial when there are multiple possible alternatives, and an agent has got to prefer to perform the simplest action.

- The utility function maps each state to a true number to see how efficiently each action achieves the goals.

- These agents aim to make rational decisions that maximize their expected utility.

Learning Agents:

- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- It starts to act with basic knowledge and then able to act and adapt automatically through learning.
- A learning agent has mainly four conceptual components, which are:
    a. **Learning element:** It is responsible for making improvements by learning from the environment.
    b. **Critic:** Learning element takes feedback from critics which describes that how well the agent is doing with respect to a fixed performance standard.
    c. **Performance element:** It is responsible for selecting external action.
    d. **Problem generator:** This component is responsible for suggesting actions that will lead to new and informative experiences.
- Hence, learning agents can learn, analyze performance, and look for new ways to improve their performance.



The Functions of an Artificial Intelligence Agent:

The functions of an agent in artificial intelligence are as follows:

- To resolve complex issues using intelligent machines.
- To decide what to do in a specific situation.
- To make conclusions and take decisions.
- The perception of dynamic environmental circumstances.
- Using logic to interpret perceptions.
- To try to change environmental conditions.

Uses of Agents:

Artificial Intelligence agents are used in various real-life applications.

Medical Evaluation:

- The surroundings are regarded as patient.
- The sensor that collects information on the patient's complaints is a computer keyboard.

- The intelligent agent uses this data to determine the best plan of action.
- Actuators used in healthcare include tests and therapies.

Automatic Vehicles:

- Various sensors are used in automatic vehicles to gather data from the surroundings.
- These consist of radar, GPS, and cameras.
- The environment in these agents could consist of people, other cars, roads, or road signs. Actions are started using a variety of devices. For instance, the car's brakes are used to stop them. Self-driving vehicles operate better with the assistance of intelligent agents.

Office Works Automation:

- The monotonous workplace tasks can be resolved using AI agents.
- Customer service and sales are two functional areas that have been automated.
- To cut operating expenses, some businesses have automated a few administrative processes.
- Office efficiency has also increased by using intelligent agents.

Agent Environment in AI

The environment is everything in the world which surrounds the agent, but it is not a part of an agent itself. An environment can be described as a situation in which an agent is present.

The environment is where the agent lives, operates, and provides the agent with something to sense and act upon it. The environment is mostly said to be non-feministic.

Features of Environment:

As for Russell and Norvig, an environment can have various features from the point of view of an agent:

1. Fully observable vs Partially Observable
2. Static vs Dynamic
3. Discrete vs Continuous
4. Deterministic vs Stochastic
5. Single-agent vs multi-agent
6. Episodic vs sequential
7. Known vs Unknown.
8. Accessible vs Inaccessible

1. Fully observable vs Partially Observable:

- For an agent sensor can sense or access the complete state of an environment at each point of time then it is **a fully observable** environment, else it is **partially observable**.
- A fully observable environment is easy as there is no need to maintain the internal state to keep track of the history of the world.
- An agent with no sensors in all environments than such an environment is called **unobservable**.

2. Deterministic vs Stochastic:

- If an agent's current state and selected action can completely determine the next state of the environment, then such an environment is called a deterministic environment.
- A stochastic environment is random in nature and cannot be determined completely by an agent.
- In a deterministic, fully observable environment, agents do not need to worry about uncertainty.

3. Episodic vs Sequential:

- In an episodic environment, there is a series of one-shot actions, and only the current percept is required for the action.
- However, in Sequential environment, an agent requires memory of past actions to determine the next best actions.

4. Single-agent vs multi-agent

- If only one agent is involved in an environment and operating by itself then such an environment is called single agent environment.
- However, if multiple agents are operating in an environment, then such an environment is called a multi-agent environment.
- The agent design problems in the multi-agent environment are different from single agent environment.

5. Static vs Dynamic:

- If the environment can change itself while an agent is deliberating, then such an environment is called a dynamic environment else it is called a static environment.
- Static environments are easy to deal with because an agent does not need to continue looking at the world while deciding on an action.

- However, for a dynamic environment, agents need to keep looking at the world at each action.
- Taxi driving is an example of a dynamic environment whereas Crossword puzzles are an example of a static environment.

6. Discrete vs Continuous:

- If in an environment there are a finite number of percepts and actions that can be performed within it, then such an environment is called a discrete environment else it is called continuous environment.
- A chess game comes under a discrete environment as there is a finite number of moves that can be performed.
- A self-driving car is an example of a continuous environment.

7. Known vs Unknown

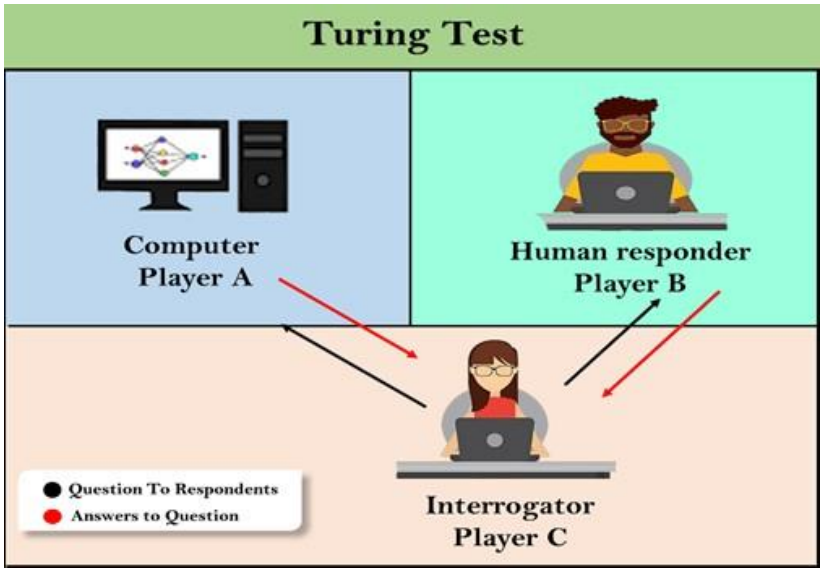
- Known and unknown are not actually a feature of an environment, but it is an agent's state of knowledge to perform an action.
- In a known environment, the results for all actions are known to the agent. While in unknown environment, agent needs to learn how it works to perform an action.
- It is quite possible that a known environment to be partially observable and an Unknown environment to be fully observable.

8. Accessible vs Inaccessible

- If an agent can obtain complete and accurate information about the state's environment, then such an environment is called an Accessible environment else it is called inaccessible.
- An empty room whose state can be defined by its temperature is an example of an accessible environment.
- Information about an event on earth is an example of Inaccessible environment.

Turing Test in AI

- In 1950, Alan Turing introduced a test to check whether a machine can think like a human or not. This test is known as the Turing Test. In this test, Turing proposed that the computer can be said to be intelligent if it can mimic human response under specific conditions.
- Turing Test was introduced by Turing in his 1950 paper, "Computing Machinery and Intelligence," which considered the question, "Can Machine think?"



The Turing test is based on a party game "Imitation game," with some modifications. This game involves three players in which one player is Computer, another player is human responder, and the third player is a human Interrogator, who is isolated from other two players and his job are to find that which player is machine among two of them.

Consider, Player A is a computer, Player B is human, and Player C is an interrogator. The interrogator is aware that one of them is a machine, but he needs to identify this based on questions and their responses.

The conversation between all players is via keyboard and screen so the result would not depend on the machine's ability to convert words as speech.

The test result does not depend on each correct answer, but only how closely its responses are like a human answer. The computer is permitted to do everything possible to force a wrong identification by the interrogator.

The questions and answers can be like:

Interrogator: Are you a computer?

Player A (Computer): No

Interrogator: Multiply two large numbers such as (256896489*456725896)

Player A: Long pause and give the wrong answer.

In this game, if an interrogator is not able to identify which is a machine and which is human, then the computer passes the test successfully, and the machine is said to be intelligent and can think like a human.

"In 1991, the New York businessman Hugh Loebner announces the prize competition, offering a $100,000 prize for the first computer to pass the Turing test. However, no AI program to till date, come close to passing an undiluted Turing test".

Chatbots to attempt the Turing test:

ELIZA: ELIZA was a Natural language processing computer program created by Joseph Weizenbaum. It was created to demonstrate the ability of communication between machines and humans. It was one of the first chatterbots, which attempted the Turing Test.

Parry: Parry was a chatterbot created by Kenneth Colby in 1972. Parry was designed to simulate a person with **Paranoid schizophrenia** (most common chronic mental disorder). Parry was described as "ELIZA with attitude." Parry was tested using a variation of the Turing Test in the early 1970s.

Eugene Goostman: Eugene Goostman was a chatbot developed in Saint Petersburg in 2001. This bot has competed in the various number of Turing

Test. In June 2012, at an event, Goostman won the competition promoted as largest-ever Turing test content, in which it convinced 29% of judges that it was a human. Goostman resembled as a 13-year old virtual boy.

The Chinese Room Argument:

There were many philosophers who really disagreed with the complete concept of Artificial Intelligence. The most famous argument in this list was "**Chinese Room**."

In the year **1980, John Searle** presented "**Chinese Room**" thought experiment, in his paper "**Mind, Brains, and Program**," which was against the validity of Turing's Test. According to his argument, "**Programming a computer may make it understand a language, but it will not produce a real understanding of language or consciousness in a computer**."

He argued that Machine such as ELIZA and Parry could easily pass the Turing test by manipulating keywords and symbol, but they had no real understanding of language. So, it cannot be described as "thinking" capability of a machine such as a human.

Features required for a machine to pass the Turing test:

- **Natural language processing:** NLP is required to communicate with Interrogator in general human language like English.
- **Knowledge representation:** To store and retrieve information during the test.
- **Automated reasoning:** To use the previously stored information for answering the questions.
- **Machine learning:** To adapt new changes and can detect generalized patterns.
- **Vision (For total Turing test):** To recognize the interrogator actions and other objects during a test.
- **Motor Control (For total Turing test):** To act upon objects if requested.

PROBLEM SOLVING AGENTS

PROBLEM-SOLVING APPROACH IN ARTIFICIAL INTELLIGENCE PROBLEMS

The **reflex agents** are known as the simplest agents because they directly map states into actions. Unfortunately, these agents fail to operate in an environment where the mapping is too large to store and learn. **Goal-based agents,** on the other hand, consider future actions and the desired outcomes.

Here, we will discuss one type of goal-based agent known as a **problem-solving agent**, which uses atomic representation with no internal states visible to the *problem-solving algorithms*.

Problem-solving agent:

The problem-solving agent performs precisely by defining problems and its several solutions.

- According to psychology, ***"a problem-solving refers to a state where we wish to reach to a definite goal from a present state or condition."***
- According to computer science, ***problem-solving is a part of artificial intelligence which encompasses several techniques such as algorithms and heuristics to solve a problem***.

Therefore, a problem-solving agent is a **goal-driven agent** and focuses on satisfying the goal.

PROBLEM DEFINITION:

To build a system to solve a particular problem, we need to do four things:

(i) **Define the problem precisely:** This definition must include specification of the initial situations and final situations which constitute (i.e.) an acceptable solution to the problem.

(ii) **Analyzing the problem**: important features have an immense (i.e) huge impact on the appropriateness of various techniques for solving the problems.

(iii) **Isolate and represent the knowledge** to solve the problem.

(iv) **Choose the best problem – solving techniques** and apply it to the problem.

<u>Steps performed by Problem-solving agent:</u>

- **Goal Formulation:** It is the first and simplest step in problem-solving. It organizes the steps/sequence required to formulate one goal out of multiple goals as well as actions to achieve that goal. Goal formulation is based on the current situation and the agent's performance measure (discussed below).
- **Problem Formulation:** It is the most important step of problem-solving which decides what actions should be taken to achieve the formulated goal.

There are the following five components involved in problem formulation:

- **Initial State:** It is the starting state or initial step of the agent towards its goal.
- **Actions:** It is the description of the possible actions available to the agent.

- **Transition Model:** It describes what each action does.
- **Goal Test:** It determines if the given state is a goal state.

  Path cost: It assigns a numeric cost to each path that follows the goal.

The problem- solving agent selects a cost function, which reflects its performance measure. Remember, **an optimal solution has the lowest path cost among all the solutions.**

Note: Initial state, actions, and **transition model** together define the **state-space** of the problem implicitly. State space of a problem is a set of all states which can be reached from the initial state followed by any sequence of actions. The state-space forms a directed map or graph where nodes are the states, links between the nodes are actions, and the path is a sequence of states connected by the sequence of actions.

- **Search:** It identifies all the best possible sequence of actions to reach the goal state from the current state. It takes a problem as an input and returns solution as its output.
- **Solution:** It finds the best algorithm out of various algorithms, which may be proven as thebest optimal solution.
- **Execution:** It executes the best optimal solution from the searching algorithms to reach thegoal state from the current state.

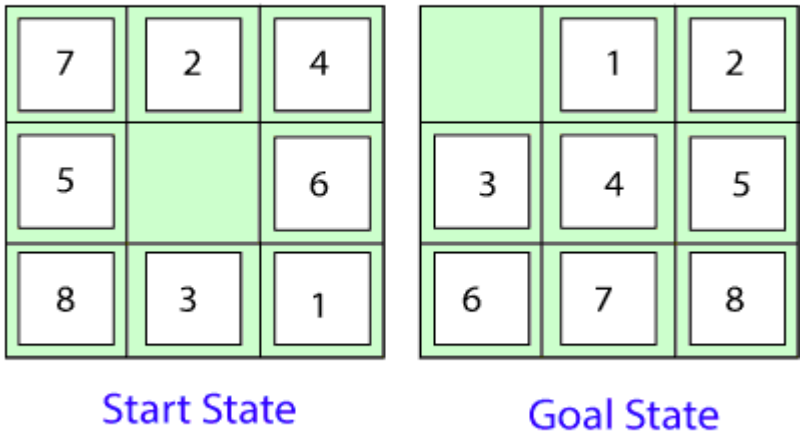
Example Problems:

Basically, there are two types of problem approaches:

- **Toy Problem:** It is a concise and exact description of the problem which is used by the researchers to compare the performance of algorithms.
- **Real-world Problem:** It is real-world based problems which require solutions. Unlike a toy problem, it does not depend on descriptions, but we can have a general formulation of the problem.

Some Toy Problems

- **8 Puzzle Problem:** Here, we have a 3×3 matrix with movable tiles numbered from 1 to 8 with a blank space. The tile adjacent to the blank space can slide into that space. The objective is to reach a specified goal state like the goal state, as shown in the below figure.
- In the figure, our task is to convert the current state into goal state by sliding digits into theblank space.

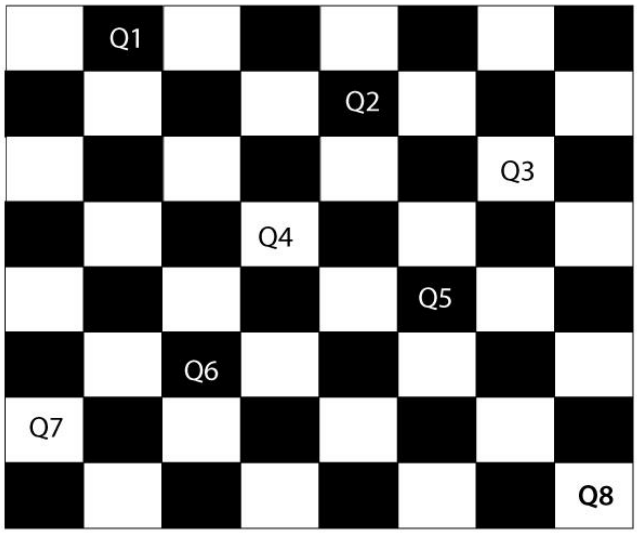


In the above figure, our task is to convert the current (Start) state into goal state by sliding digits into the blank space.

The problem formulation is as follows:

- **States:** It describes the location of each numbered tile and the blank tile.
- **Initial State:** We can start from any state as the initial state.
- **Actions:** Here, actions of the blank space are defined, i.e., either **left, right, up or down**
- **Transition Model:** It returns the resulting state as per the given state and actions.
- **Goal test:** It identifies whether we have reached the correct goal-state.
- **Path cost:** The path cost is the number of steps in the path where the cost of each step is 1. **Note:** The 8-puzzle problem is a type of **sliding-block problem** which is used for testing new search algorithms in artificial intelligence.
- **8-queens problem:** The aim of this problem is to place eight queens on a chessboard in an order where no queen may attack another. A queen can attack other queens either **diagonally or in the same row and column.**

From the following figure, we can understand the problem as well as its correct solution.



It is noticed from the above figure that each queen is set onto the chessboard in a position where no other queen is placed diagonally, in the same row or column. Therefore, it is one right approach to the 8-queens problem.

For this problem, there are two main kinds of formulation:

1. **Incremental formulation:** It starts from an empty state where the operator extends a queen at each step.

The following steps are involved in this formulation:

- **States:** Arrangement of any 0 to 7 queens on the chessboard.
- **Initial State:** An empty chessboard
- **Actions:** Add a queen to any empty box.
- **Transition model:** Returns the chessboard with the queen added in a box.
- **Goal test:** Checks whether 8-queens are placed on the chessboard without any attack.

- **Path cost:** There is no need for path cost because only final states are counted. In this formulation, there is approximately **1.8 x 10^{14}** possible sequence to investigate.

2. **Complete-state formulation:** It starts with all the 8-queens on the chessboard and moves themaround, saving from the attacks.

The following steps are involved in this formulation

- **States:** Arrangement of all the 8 queens one per column with no queen attacking the otherqueen.
- **Actions:** Move the queen at the location where it is safe from the attacks.

This formulation is better than the incremental formulation as it reduces the state space from **1.8 x10^{14}** to **2057**, and it is easy to find the solutions.

Some Real-world problems:

- **Traveling salesperson problem (TSP):** It is a **touring problem** where the salesman can visit each city only once. The objective is to find the shortest tour and sell out the stuff in each city.
- **VLSI Layout problem:** In this problem, millions of components and connections are positioned on a chip in order to minimize the area, circuit-delays, stray-capacitances, and maximizing the manufacturing yield.

  The layout problem is split into two parts:

  - **Cell layout:** Here, the primitive components of the circuit are grouped into cells, each performing its specific function. Each cell has a fixed shape and size. The task is to place the cells on the chip without overlapping each other.

- **Channel routing:** It finds a specific route for each wire through the gaps betweenthe cells.

Protein Design: The objective is to find a sequence of amino acids which will fold into 3D protein having a property to cure some disease.

Searching for solutions:

We have seen many problems. Now, there is a need to search for solutions to solve them.

In this section, we will understand how searching can be used by the agent to solve a problem.

For solving different kinds of problems, an agent makes use of different strategies to reach the goal by searching the best possible algorithms. This process of searching is known as **a search strategy.**

Search Algorithms in Artificial Intelligence

Search algorithms are one of the most important areas of Artificial Intelligence. This topic will explain all about the search algorithms in AI.

Problem-solving agents:

In Artificial Intelligence, Search techniques are universal problem-solving methods. **Rational agents** or **Problem-solving agents** in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are goal-based agents and use atomic representation. In this topic, we will learn various problem-solving search algorithms.

Search Algorithm Terminologies:

- **Search:** Searching is a step-by-step procedure to solve a search-problem in each search space. A search problem can have three main factors:
  a. **Search Space:** Search space represents a set of possible solutions, which system may have.
  b. **Start State:** It is a state from where the agent begins **the search**.
  c. **Goal test:** It is a function which observes the current state and returns whether the goal state is achieved or not.
- **Search tree:** A tree representation of search problem is called Search tree. The root of the search tree is the root node which corresponds to the initial state.
- **Actions:** It gives a description of all the available actions to the agent.
- **Transition model:** A description of what each action does can be represented as a transition model.
- **Path Cost:** It is a function which assigns a numeric cost to each path.
- **Solution:** It is an action sequence which leads from the start node to the goal node.

- **Optimal Solution:** If a solution has the lowest cost among all solutions.

Properties of Search Algorithms:

The following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

Completeness: A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.

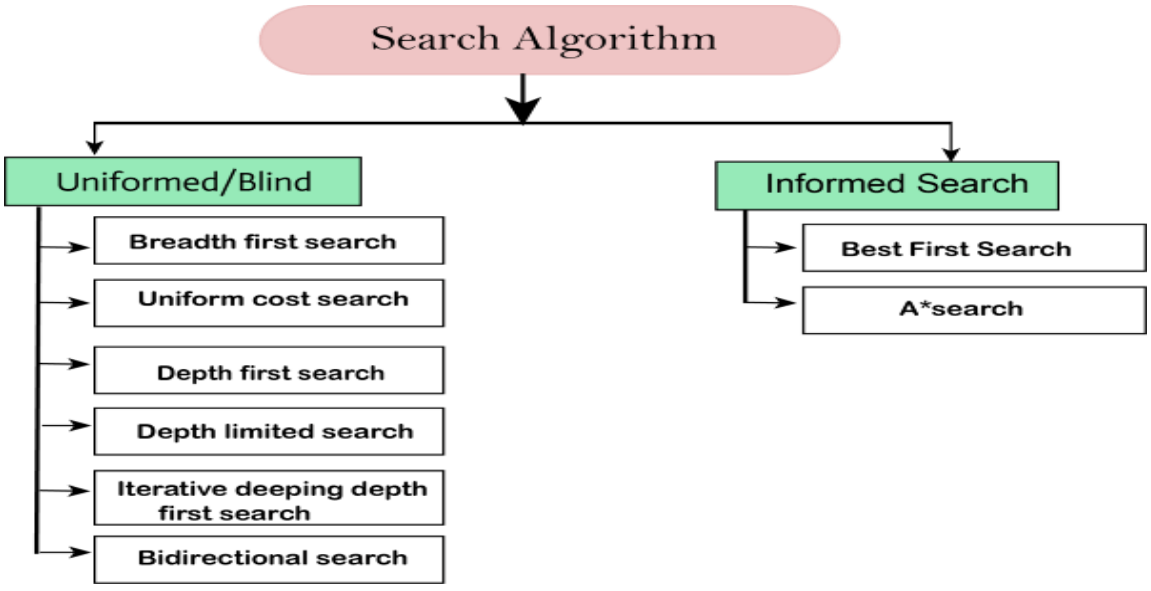
Optimality: If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution for is said to be an optimal solution.

Time Complexity: Time complexity is a measure of time for an algorithm to complete its task.

Space Complexity: It is the maximum storage space required at any point during the search, as the complexity of the problem.

Types of search algorithms

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Uninformed/Blind Search:

The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes. Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called blind search. It examines each node of the tree until it achieves the goal node.

It can be divided into five main types:

- Breadth-first search
- Uniform cost search
- Depth-first search
- Iterative deepening depth-first search
- Bidirectional Search

Informed Search:

Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a Heuristic search.

A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.

Informed search can solve many complex problem which cannot be solved in another way.

An example of informed search algorithms is a traveling salesman problem.

1. Greedy Search
2. A* Search

Uninformed Search Algorithms:

Uninformed search is a class of general-purpose search algorithms which operate in brute force-way. Uninformed search algorithms do not have additional information about state or search space other than how to traverse the tree, so it is also called blind search.

The following are the various types of uninformed search algorithms:

1. Breadth-first Search
2. Depth-first Search
3. Depth-limited Search
4. Iterative deepening depth-first search
5. Uniform cost search
6. Bidirectional Search

1. Breadth-first Search:

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO queue data structure.

Advantages:

- BFS will provide a solution if any solution exists.
- If there are more than one solution for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

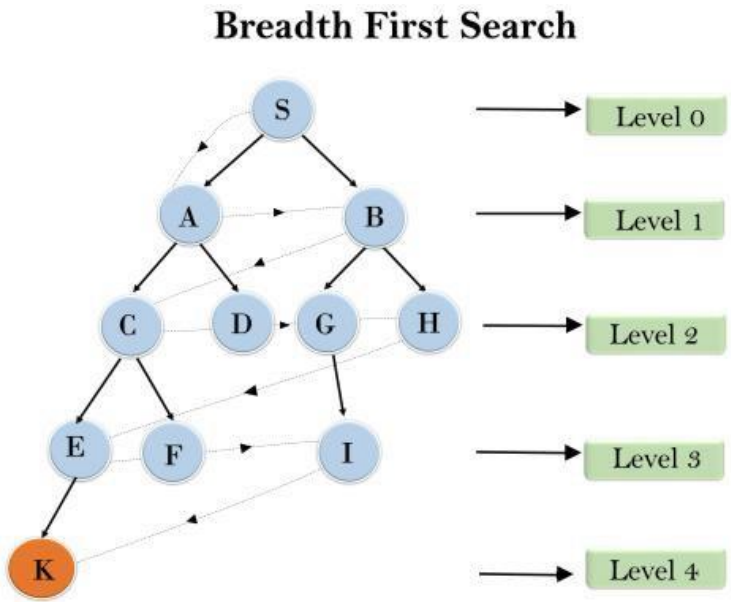
Disadvantages:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

Example:

In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

1. S---> A--->B---->C--->D---->G--->H--->E---->F---->I---- >K



Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.

$T(b) = 1+b^2+b^3+\ldots\ldots+ b^d = O(b^d)$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the node.

2. Depth-first Search

- Depth-first search isa recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is like the BFS algorithm.

Advantage:

- DFS requires very little memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach the goal node than BFS algorithm (if it traverses in the right path).

Disadvantages:

- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep searching and sometimes it may go to the infinite loop.

Example:

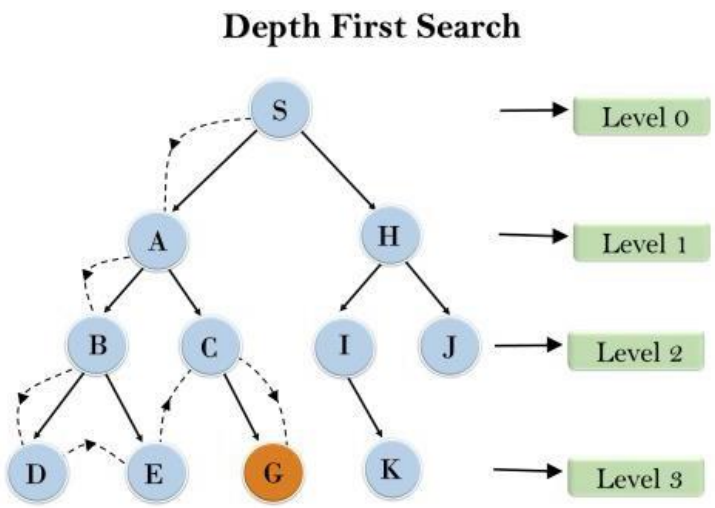
In the search below, we have shown the flow of depth-first search, and it will follow the order as:

Root node--->Left node ----> right node.

It will start searching from root node S, and traverse A, then B, then D and E. After traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.

Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:



Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$T(n)= 1+ n^2+ n^3 +\ldots\ldots\ldots+ n^m=O(n^m)$

Where, m= maximum depth of any node and this can be much larger than d (Shallowest solution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **O(bm)**.

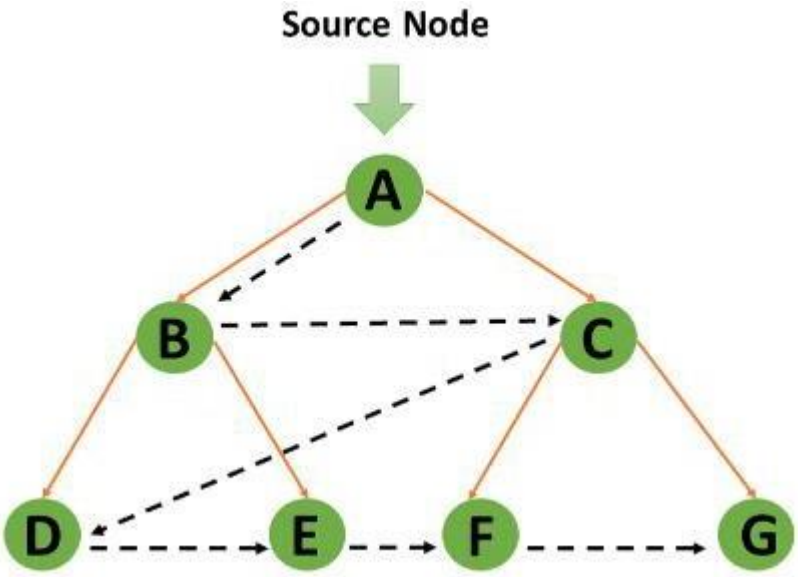
Optimal: DFS search algorithm is non-optimal, as it may generate many steps or high cost to reach the goal node.

Breadth First Search (BFS):

BFS is a fundamental graph traversal algorithm that explores all the vertices in a graph in a level-by-level manner. It starts at a specified vertex and visits all its neighbors before moving on to the next level of neighbors. BFS is commonly used in various applications, including pathfinding, connected components, and shortest path problems in graphs.

According to the BFS, you must traverse the graph in a breadth wise direction:

- To begin, move horizontally and visit all the current layer's nodes.
- Continue to the next layer.



Breadth-First Search uses a queue data structure to store the node and mark it as "visited" until it marks all the neighboring vertices directly related to it. The queue operates on the First in First out (FIFO) principle, so the node's neighbors will be viewed in the order in which it inserts them in the node, starting with the node that was inserted first.

Algorithm for BFS:

1. Initialization:

Enqueue the starting node into a queue and mark it as visited.

2. Exploration:

While the queue is not empty:

Dequeue a node from the queue and visit it (e.g., print its value).

For each unvisited neighbor of the dequeued node:

Enqueue the neighbor into the queue.

Mark the neighbor as visited.

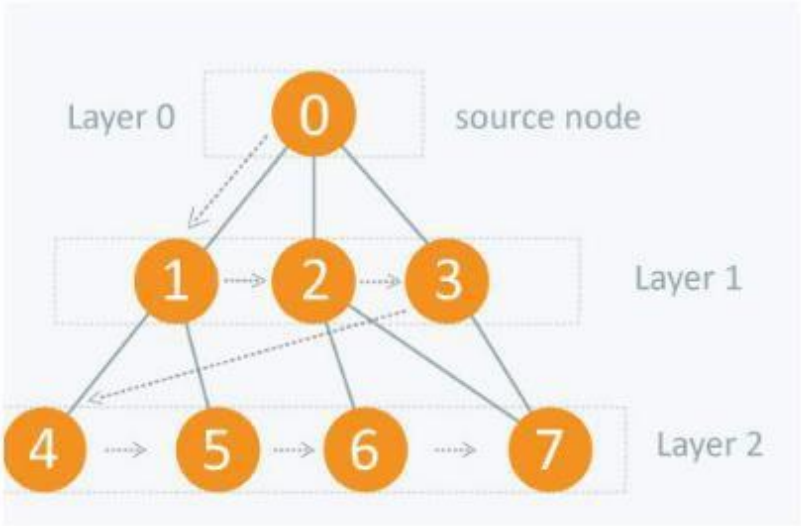
3. Termination:

Repeat step 2 until the queue is empty.

This algorithm ensures that all nodes in the graph are visited in a breadth-first manner, starting from the initial node.

How Does BFS Work?

Starting from the root, all the nodes at a particular level are visited first, and then the nodes of the next level are traversed until all nodes are visited. A queue is used to keep track of adjacent unvisited nodes at each level.



The above tree diagram contains three layers, which are numbered from 0 to 2.

- We are allowed to use any node as our source node as per the law. However, in this case, we can use 0 as our source node.
- Then we explore breadth wise and find the nodes which are adjacently connected to our source node.
- Then we must come down to layer two and find the relative nodes that are adjacent to the layer 1 nodes.
- If you select the alternative source node, this order will change. And that's how the straightforward BFS algorithm operates.

Example Illustration

Let’s understand the working of BFS with an example. Consider the following directed graph:

```
  A
 / \
B   C
|   |
D   G
|   |
E   F
```

- Initially, the queue and visited arrays are empty.
- Push node A into the queue and mark it visited.
- Dequeue A, visit it, and enqueue its neighbors B and C.
- Dequeue B, visit it, and enqueue its children D and E.
- Dequeue C, visit it, and enqueue its children D and G.
- Continue this process until all nodes are visited.

Advantages of BFS:

1. **Completeness:** If there is a solution, BFS is guaranteed to find it.
2. **Optimality:** Finds the shortest path (minimal number of edges) to the goal.
3. **Simplicity:** One of the simplest search strategies.

Disadvantages of BFS:

1. **Memory Requirement:** BFS consumes large memory space due to the queue.
2. **Time Complexity:** In the worst case, BFS explores all vertices and edges, resulting in a time complexity of O(V + E).

Remember, BFS is a powerful tool for exploring graphs, but its memory usage can be a concern in large graphs.

Time Complexity of BFS:

b -> b is a node at every state

d -> depth of shallowest node (goal node)

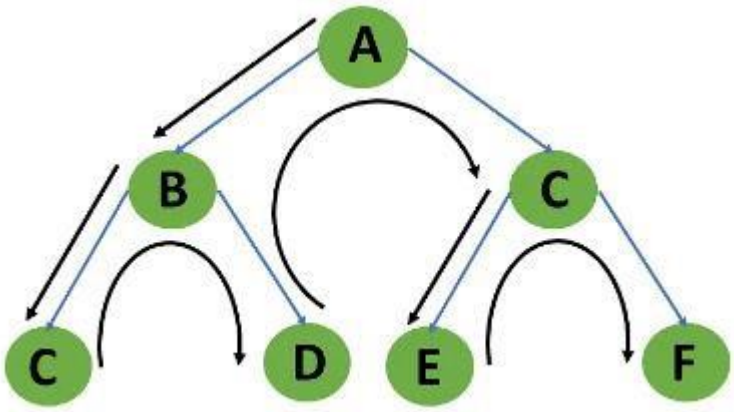
$T(b)= 1+b^2+b^3+\dots\dots\dots\dots+b^d$

$=O(b^d)$

$s(b) = O(b^d)$

Depth First Search (DFS)

Depth First Search (DFS) is a fundamental algorithm used in graph theory to traverse or search through the nodes of a graph. DFS is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at a selected node (root if it's a tree) and explores as far as possible along each branch before backtracking. When a dead-end occurs in any iteration, the Depth First Search (DFS) method traverses a network in a deathward motion and uses a stack data structure to remember to acquire the next vertex to start a search. To visit the next node, pop the top node from the stack and push all its nearby nodes into a stack.



Topological sorting, scheduling problems, graph cycle detection, and solving puzzles with just one solution, such as a maze or a sudoku puzzle, all employ depth-first search algorithms. Other applications include network analysis, such as determining if a graph is bipartite.

Algorithm:

1. Create a stack with the total number of vertices in the graph as the size.

2. Choose any vertex as the traversal's beginning point. Start at the root node
3. Mark the node as visited and push it onto the stack.
4. While the stack is not empty:
   - Pop a node from the stack.
   - If the node has unvisited adjacent nodes, mark the adjacent node as visited and push it onto the stack.
   - If the node has no unvisited adjacent nodes, backtrack.

Example Graph:

Imagine we have the following graph:



DFS Algorithm Steps:

1. Start at the root node (1).
2. Visit node 1 and push it onto the stack.
3. Go to an unvisited adjacent node (2), visit it, and push it onto the stack.
4. Continue to node 4, then to node 6, visiting each and pushing them onto the stack.
5. Since node 6 has no unvisited adjacent nodes, backtrack to node 4.
6. Backtrack to node 2, and since it has no unvisited adjacent nodes, backtrack to node 1.

7. Visit node 3 from node 1, then node 5, and finally node 7, visiting each one.

DFS Visiting Order:

The nodes are visited in the following order: 1, 2, 4, 6, 3, 5, 7.

This is a typical depth-first traversal, where we go as deep as possible along each branch before backtracking to explore new branches. The DFS algorithm is particularly useful in scenarios where we want to explore all possible paths or configurations, such as in puzzle solving, game state exploration, and more.

Advantages:

1. **Memory Efficiency:** DFS requires less memory as it stores only the nodes on the current path and their siblings3.
2. **Solution Depth:** DFS can find a solution without exploring much of the search space if the solution is deep and not far from the root node4.

Disadvantages:

1. **Non-Optimal Solution:** DFS may not find the shortest path to a solution5.
2. **Infinite Loops:** It can get stuck in infinite loops if the graph has cycles and precautions are not taken5.

Time Complexity: If you can access each node in **O(1)** time, then with branching factor of b and max depth of m, the total number of nodes in this tree would be worst case **= $1 + b + b^2 + \ldots + b^{m-1}$**. Using the formula for summing a geometric sequence (or even solving it ourselves) tells that this sums to **= $(b^m - 1)/(b - 1)$,** resulting in total time to visit each node proportional to b^m. Hence the complexity **= $O(b^m)$.**

Space Complexity: The length of longest path = m. For each node, you must store its siblings so that when you have visited all the children, and you come back to a parent node, you can know which sibling to explore

next. For m nodes down the path, you will have to store b nodes extra for each of the m nodes. That's how you get an **$O(b^m)$** space complexity. DFS is a powerful algorithm that forms the basis of many other complex graph algorithms and is widely used in applications such as solving puzzles, scheduling problems, and network analysis.

Applications of DFS:

Depth First Search (DFS) is a versatile algorithm with a wide range of applications in computer science and beyond. Here are some notable applications:

1. **Cycle Detection:** DFS can be used to detect cycles in a graph, which is crucial in many applications, such as deadlock detection in operating systems.
2. **Path Finding:** DFS can find a path between two given vertices, which is useful in routing and navigation systems.
3. **Topological Sorting:** This is used for scheduling jobs from the given dependencies among jobs, such as instruction scheduling, logic synthesis, and resolving symbol dependencies in linkers.
4. **Testing Bipartiteness:** DFS can test whether a graph is bipartite, which is useful in modeling problems that require partitioning, such as scheduling and assigning tasks.
5. **Finding Strongly Connected Components:** In directed graphs, DFS helps identify strongly connected components, which can be applied in understanding the structure of networks.
6. **Solving Puzzles:** DFS is suitable for puzzles that have only one solution, like mazes, as it explores exhaustively until a solution is found.
7. **Web Crawlers:** DFS can be used to navigate the web systematically for web indexing by search engines.
8. **Maze Generation:** DFS can generate random mazes, which is used in game development and recreational mathematics.

9. **Model Checking:** It’s used in verifying that a model of a system meets a certain set of properties.
10. **Backtracking Algorithms:** DFS is the foundation of backtracking algorithms, which are used in constraint satisfaction problems like solving Sudoku.

These applications leverage the exhaustive searching capability of DFS, making it a fundamental tool in algorithms and data structure manipulation.

Hill Climbing Search

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak where no neighbor has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing mathematical problems. One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.
- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.
- A node of hill climbing algorithm has two components which are state and value.
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

Features of Hill Climbing:

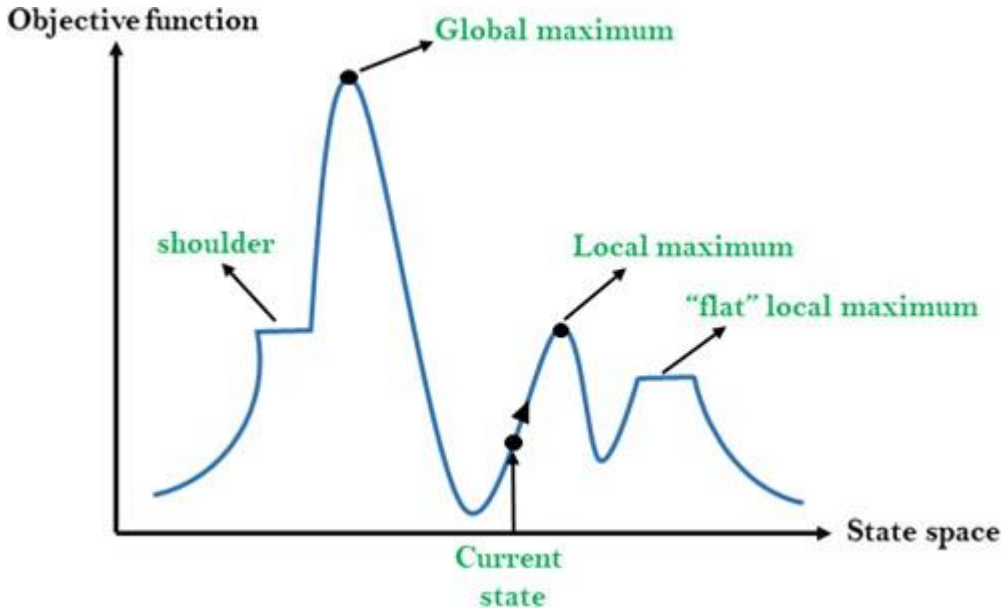
The following are some main features of Hill Climbing Algorithm:

- **Generate and Test variant:** Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce feedback which helps to decide which direction to move in the search space.
- **Greedy approach:** Hill-climbing algorithm search moves in the direction which will give the best outcome, which optimizes the cost.
- **No backtracking:** It does not backtrack the search space, as it does not remember the previous states. i.e., the algorithm cannot go back to its old path.

'

State-space Diagram for Hill Climbing:

- The state-space landscape / diagram is a graphical representation of the hill-climbing algorithm which shows a graph between various states of algorithm and Objective function/Cost.
- On Y-axis we have taken the function which can be an objective function or cost function, and state-space on the x-axis. If the function on Y-axis is cost then, the goal of search is to find the global minimum and local minimum. If the function of Y-axis is Objective function, then the goal of the search is to find the global maximum and local maximum.



Different regions in the state space landscape:

Local Maximum: Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.

Global Maximum: Global maximum represents the highest peak in the entire state space diagram, that is the best possible state of state space landscape. It has the highest value of objective function.

Current state: It is a state in a landscape diagram where an agent is currently present during the search.

Flat local maximum/plateau: It is a flat space in the landscape where all the neighbor states of current states have the same cost function.

Shoulder: It is a plateau region which has an uphill edge.

Types of Hills Climbing Algorithm:

- Simple hill Climbing:
- Steepest-Ascent hill-climbing:
- Stochastic hill Climbing:

1. Simple Hill Climbing:

Simple hill climbing is the simplest way to implement a hill climbing algorithm. **It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and sets it as a current state**. It only checks its successor state, and if it finds better than the current state, then move else be in the same state.

This algorithm has the following features:

- Less time consuming
- Less optimal solution and the solution is not guaranteed

2. Steepest-Ascent hill climbing:

- The steepest Ascent algorithm is a variation of simple hill climbing algorithms. This algorithm examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbors.

3. Stochastic hill climbing:

- Stochastic hill climbing does not examine all its neighbors before moving. Rather, this search algorithm selects one neighbor node at random and decides whether to choose it as a current state or examine another state.

Algorithm for Simple Hill Climbing:

- **Step 1:** Evaluate the initial state, if it is goal state then return success and stop.
- **Step 2:** Loop Until a solution is found or there is no new operator left to apply.
- **Step 3:** Select and apply an operator to the current state. Then we get the new state.
- **Step 4:** Check new state:
    a. If it is a goal state, then return success and quit.
    b. Else if it is better than the current state then assigns new state as a current state.
    c. Else if not better than the current state, then return to step2.
    d. Until the process of getting the goal state.
- **Step 5:** Exit.

Or

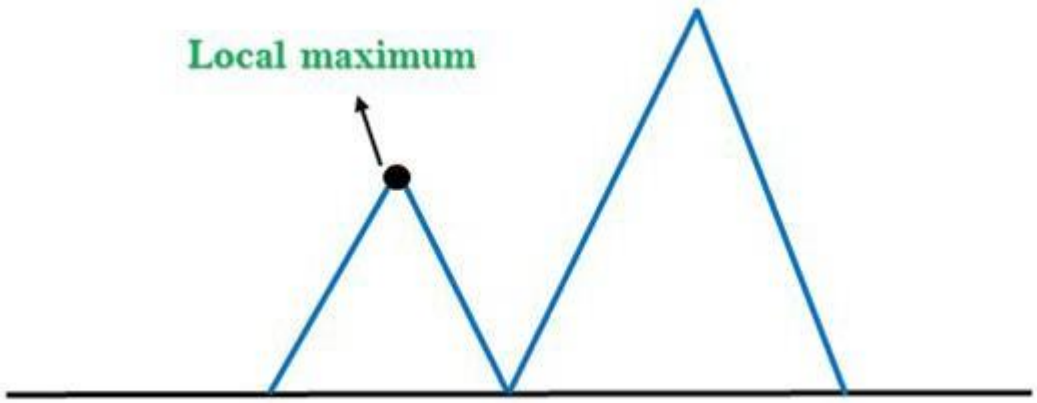
Function HILL-CLIMBING (problem) returns to a state that is a local maximum.

```
Current <- problem.INITIAL
  While true do
      Neighbor<- a highest -valued successor state of current
      If VALUE(neighbor)<= VALUE(current) then return current
      Current<-neighbor
```

Problems in Hill Climbing Algorithm:

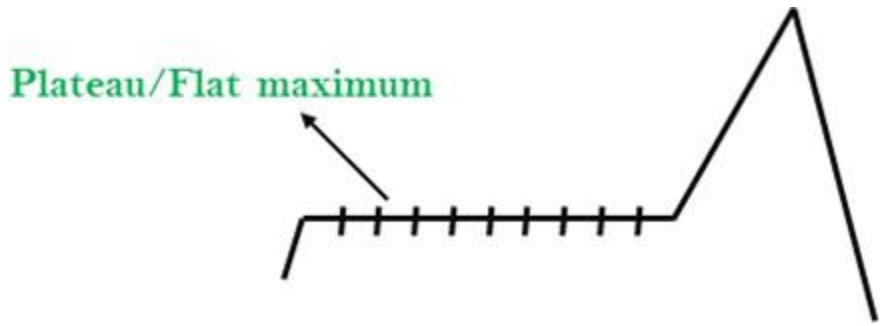
1. Local Maximum: A local maximum is a peak state in the landscape, which is better than each of its neighboring states, but there is another state also present which is higher than the local maximum.

Solution: Backtracking technique can be a solution of the local maximum in state space landscape. Create a list of the promising path so that the algorithm can backtrack the search space and explore other paths as well.



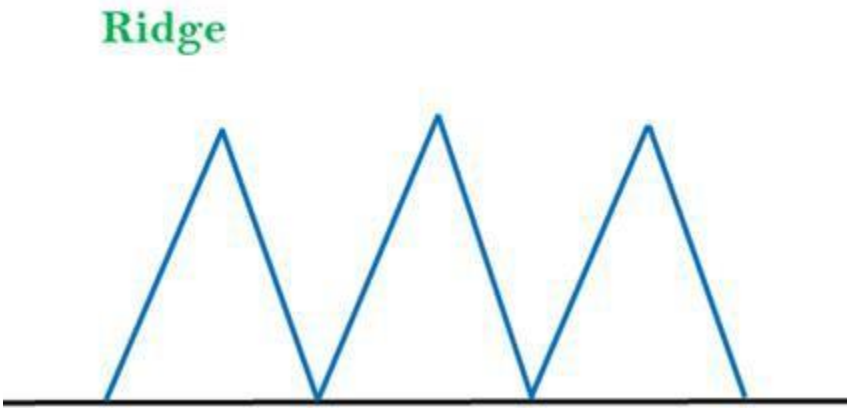
2. Plateau: A plateau is the flat area of the search space in which all the neighbor states of the current state contain the same value, because this algorithm does not find any best direction to move. A hill-climbing search might be lost in the plateau area.

Solution: The solution for the plateau is to take big steps or very little steps while searching, to solve the problem. Randomly select a state which is far away from the current state so it is possible that the algorithm could find a non-plateau region.



3. Ridges: A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.

Solution: With the use of bidirectional search, or by moving in different directions, we can improve this problem.



Simulated Annealing Search

- The simulated annealing algorithm is based upon physical annealing in real life.
- Physical annealing is the process of heating up a material until it reaches an annealing temperature and then.
- It will be cooled down slowly to change the material to a desired structure.
- When the material is hot, the molecular structure is weaker and is more susceptible to change.
- When the material cools down, the molecular structure is harder and less susceptible to change.
- Thermal Dynamics Equation calculates the probability that the Energy Magnitude will increase.

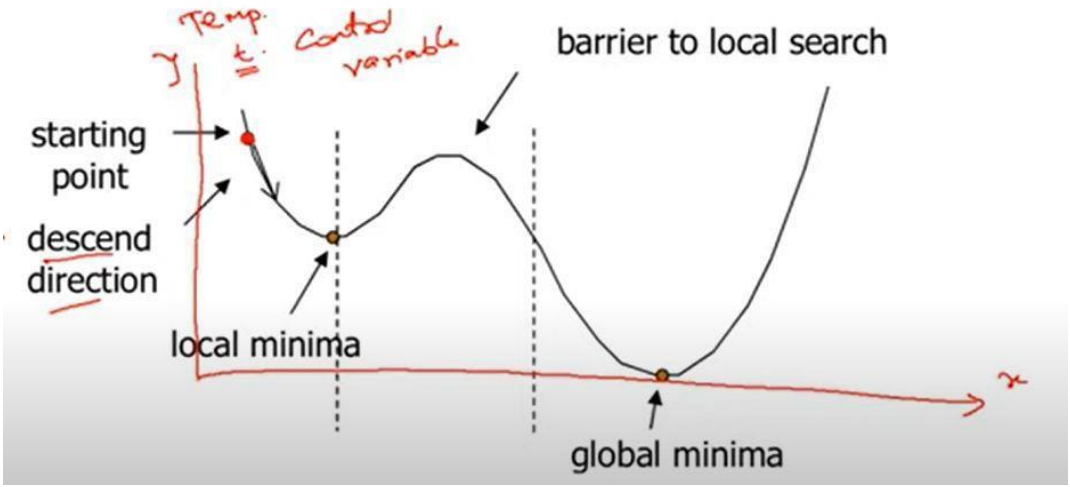
$$P(\Delta E) = e^{-\frac{\Delta E}{k*t}}$$

- Where ΔE - Energy Magnitude
- t - temperature
- k - Boltzmann constant.

Simulated Annealing:

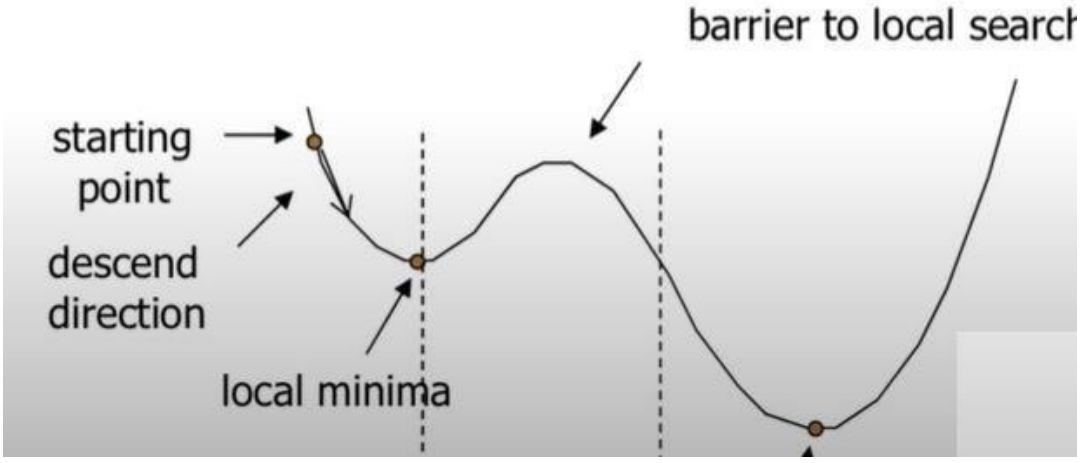
- Simulated Annealing is a stochastic global search optimization algorithm, and it is a modified version of stochastic hill climbing.
- the algorithm appropriate for nonlinear objective functions where other local search algorithms do not operate well.
- This simulated -annealing solution is to start by shaking hard (i.e., at a high temperature) and

- then gradually reduce the intensity of the shaking (i.e., lower the temperature).
- simulated Annealing (SA) is very useful for situations where there are a lot of local minima.



simulated Annealing – Example ping-pong ball:

- imagine the task of getting a ping-pong ball into the deepest crevice in a very bumpy surface.
- If we just let the ball roll, it will come to rest at a local minimum.
- The trick is to shake just hard enough, to bounce the ball out of local minima then the ball will reach the global minimum.



- simulated annealing improves this strategy through the introduction of two tricks.
- This algorithm picks a random move instead of picking the best move.
- If the move improves the result, then it accepts this random move, otherwise it accepts the move with some probability less than 1.
- Based on annealing, the algorithm uses the equation

- Based on annealing, the algorithm uses the equation

$$e^{-\Delta D/T} > R(0, 1),$$

- Where ΔD is Change of distance,
- T is synthetic temperature (control Variable) and
- R(0,1) is a random number in the interval [0,1].

Simulated Annealing Algorithm

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state
  current ← *problem*.INITIAL
  for $t = 1$ **to** ∞ **do**
    $T \leftarrow schedule(t)$
    if $T = 0$ **then return** *current*
    next ← a randomly selected successor of *current*
    $\Delta E \leftarrow$ VALUE(*current*) – VALUE(*next*)
    if $\Delta E > 0$ **then** *current* ← *next*
    else *current* ← *next* only with probability $e^{-\Delta E/T}$

- Simulated annealing was used to solve VLSI layout problems
- It has been applied widely to factory scheduling and
- Other large-scale optimization tasks.

Local Search in Continuous Space

Local search algorithms play a crucial role in optimization problems, especially when finding an exact solution is computationally expensive or impossible. In continuous spaces, where the solution space is not discrete but rather a continuum, local search techniques are particularly valuable.

Local search is an optimization technique that iteratively explores the solution space by making small adjustments to an initial solution.

Unlike global search methods that explore multiple paths simultaneously, local search focuses on a single current node and its neighbors.

The goal is to find an optimal solution by iteratively improving the current solution.

A local search is first conducted in the continuous space until a local optimum is reached. It then switches to a discrete space that represents a discretization of the continuous model to find an improved solution from there.

The process continues, switching between the two problem formulations until no further improvement can be found in either.

To perform local search in continuous state space we need techniques from calculus.

The main technique to find a minimum is called gradient descent.

Gradient descent:

A gradient measures how much the output of a function changes if you change the inputs a little bit.

In machine learning, a gradient is a derivative of a function that has more than one input variable. known as the slope of a function in mathematical terms, the gradient simply measures the change in all weights about the change in error.

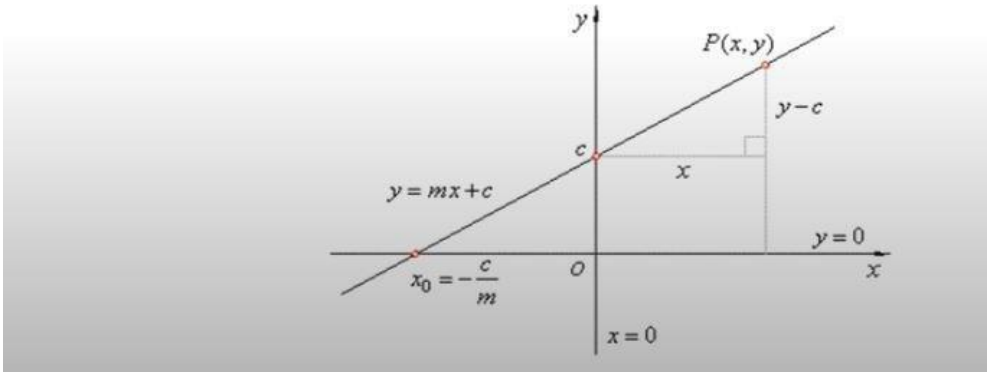
Gradient descent is an iterative optimization algorithm to find the minimum of a function.

Gradient descent is an optimization algorithm for finding a local minimum of a differentiable function. gradient descent is simply used in machine learning to find the values of function's parameters (coefficients) that minimize a cost function as far as possible.

Linear Regression:

In statistics, linear regression is a linear approach to modelling the relationship between a dependent variable and one or more independent variables.

Y = m X + c



Loss function:

The loss is the error in our predicted value of m and c. our goal is to minimize this error to obtain the most accurate value of m and c.

Mean Squared Error function to calculate the loss:

Step 1: find the difference between the actual y and predicted y value(y=mx+c), for a given x.

Step 2: square this difference.

Step 3: find the meaning of the squares for every value in X.

- Mean Squared Error Equation

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - \bar{y}_i)^2$$

- Substituting the value of $\bar{y}_i$

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - (mx_i + c))^2$$

Gradient Descent Algorithm:

Applying gradient descent to m and c and approach it step by step:

Step 1: initially let m=0 and c=0. Let L be our learning rate. This controls how much the value of m changes with each step. L could be a small value like 0.0001 for good accuracy.

Step 2: calculate the partial derivative of the loss function with respect to m, and plug in the current values of x,y , m and c in it to obtain the derivative value D.

$$D_m = \frac{1}{n}\sum_{i=0}^{n} 2(y_i - (mx_i + c))(-x_i)$$

$$D_m = \frac{-2}{n}\sum_{i=0}^{n} x_i(y_i - \bar{y}_i)$$

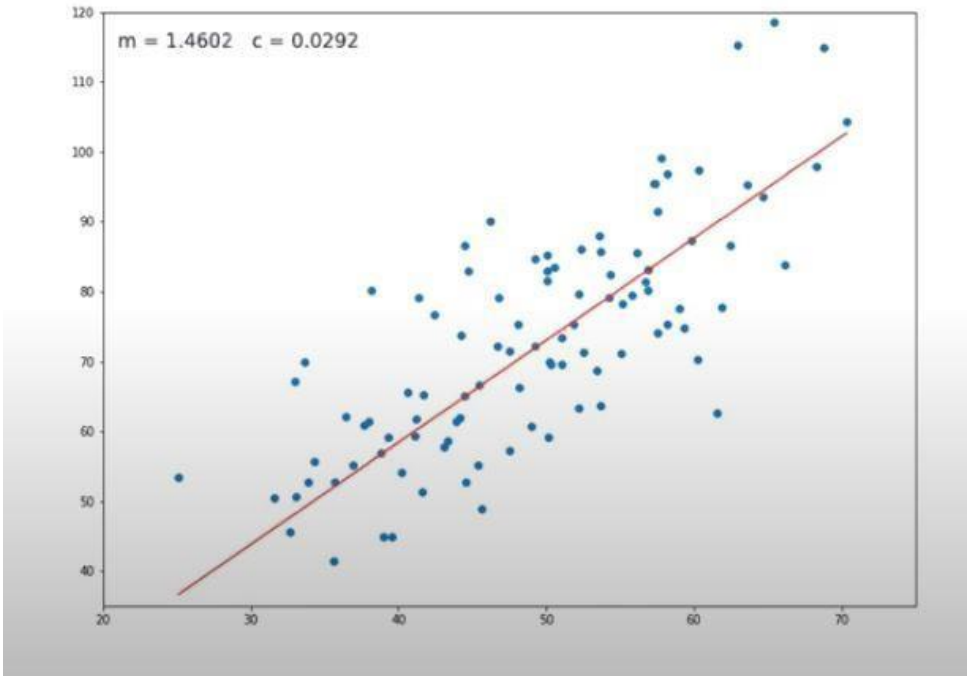
Step 3: Now we update the current value of m and c using the following equation:

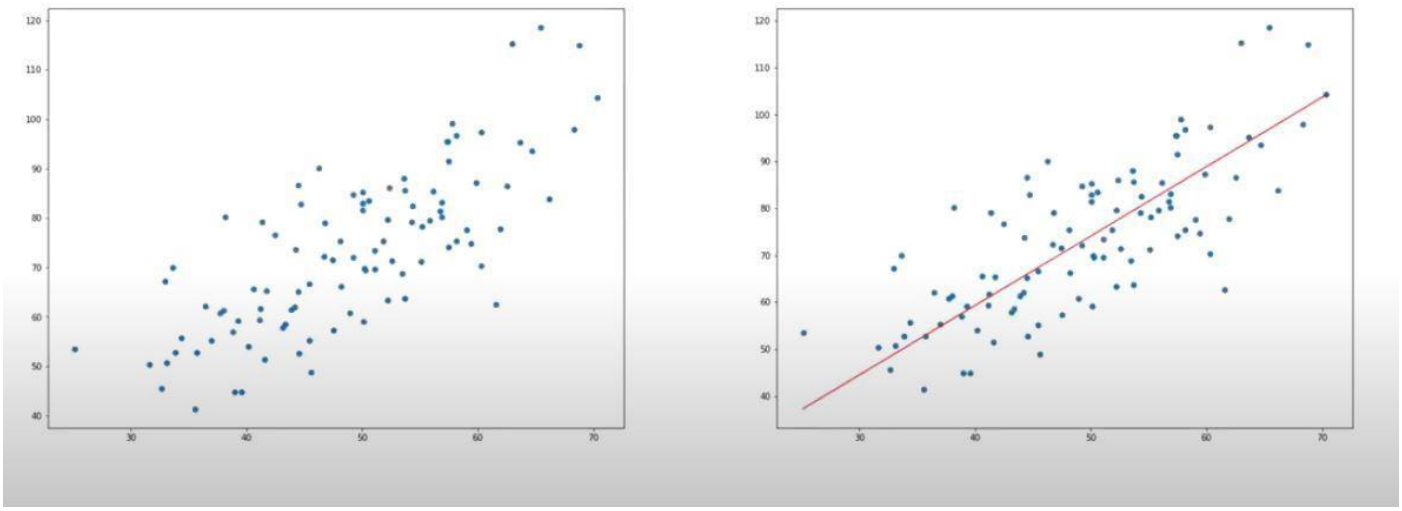
$$m = m - L \times D_m$$

$$c = c - L \times D_c$$

Step 4: we repeat this process until our loss function is a very small value or ideally 0(which means 0 error or 100% accuracy). The value of m and c that we are left with now will be the optimum values.

The value of m and c are updated at each iteration to get the optimal solution.





Characteristics of Local Search in Continuous Spaces:

Iterative Process: Local search algorithms repeatedly modify the current solution to explore nearby solutions.

Single Current Node: Instead of branching into multiple paths, local search operates on a single node and its neighbors.

Infinite Branching Factors: Continuous spaces have infinite possibilities, making local search adaptable to various problem domains.

Local Search Algorithms for Continuous Spaces:

Hill Climbing:

Start with an initial solution and iteratively move to a neighboring solution with a higher objective value. Works well for continuous state and action spaces.

Example: Climbing a hill by always moving uphill.

Simulated Annealing:

Inspired by annealing in metallurgy, it probabilistically explores the solution space.

Allows occasional downhill moves to escape local optima.

Widely used in optimization problems.

Tabu Search, Genetic Algorithms, and other variants can also be adapted for continuous spaces.

Challenges in Local Search:

Local Maxima: Getting stuck in suboptimal solutions.

Plateaus: Flat regions where small adjustments don't lead to improvement.

Applications for Local Search in Continuous Spaces:

Engineering: Optimal design parameters, circuit layout, and resource allocation.

Economics: Portfolio optimization, pricing strategies, and supply chain management.

AI: Neural network training, hyperparameter tuning, and reinforcement learning.

Local search in continuous spaces is a powerful tool for solving complex optimization problems. Its adaptability, efficiency, and ability to handle infinite solution spaces make it indispensable in AI and beyond.